

La refactorización:

Utiliza una serie de patrones, de aplicación sobre el código fuente

¿Cuál no es un patrón de refactorización?

Eliminar parámetros de un método

¿Qué afirmación sobre control de versiones es correcta?

El repositorio centraliza el almacenamiento de los datos

¿Qué herramienta no es una herramienta de Control de Versiones?

PMD

¿Cuál de las siguientes no es una tarea básica de la Gestión de Configuraciones del Software?

Gestión del repositorio

La Gestión de Configuraciones:

Es un conjunto de actividades desarrolladas para gestionar los cambios a lo largo del ciclo de vida del software

¿Qué cliente de Gestión de Versiones no incorpora NetBeans?

VS Team Foundation

Un comentario en formato JavaDoc.

Comienza por `/**` y terminan por `*/`

1. Refactorización (Conceptos Clave)

*

****Definición:**** Es una técnica que consiste en modificar la estructura interna del código para hacerlo más fácil de entender y de mantener, ****sin modificar su comportamiento o funcionalidad****. No corrige errores ni añade funcionalidades.

*** **Limitaciones (Cuándo es difícil o no se debe refactorizar):****

*

***Bases de datos:** Tienen muchas interdependencias y modificarlas es muy costoso.

*

***Interfaces públicas:** Si cambias una interfaz pública (ej. renombrar un método), debes cambiar todas las llamadas a él en todo el proyecto.

*

Errores de diseño graves:** Si el código no funciona o el diseño es un desastre, *no se refactoriza, se reescribe desde cero****.

* **Patrones de Refactorización más comunes:**

*

****Rename (Renombrado):**** Cambiar un nombre para que sea más significativo (actualiza todas las referencias).

*

****Extract Method (Extraer método):**** Sustituir un bloque de sentencias largo o repetitivo por la llamada a un nuevo método.

*

****Encapsular campos:**** Convertir campos en privados y usar **getters** y **setters**.

*

****Move (Mover):**** Mover una clase de un paquete a otro.

*

****Inline:**** Sustituir la referencia a una variable o método por el código en una sola línea.

*

****Extract Interface / Superclass:**** Extraer métodos comunes de clases para crear una interfaz o superclase padre.

2. "Bad Smells" (Malos olores del código)

No son errores de código (bugs), sino "síntomas" de que el diseño se puede mejorar.

¡Atención! Suelen caer en los test relacionando el nombre con su definición:

*

****Duplicated code (Código duplicado):**** Bloques idénticos en varias partes (se soluciona extrayendo a un método).

* ****Long Method (Método largo):**** Un método que hace demasiadas cosas. Deben ser cortos y hacer una sola cosa.

*

****Large class (Clase grande):**** Clase con demasiados métodos/atributos y responsabilidades.

*

****Long parameter list:**** Métodos con demasiados parámetros (se soluciona pasando un objeto en su lugar).

*

****Divergent change (Cambio divergente):**** Cuando una misma clase sufre modificaciones frecuentes por motivos muy distintos.

*** **Shotgun surgery (Cirugía a tiros):**** Lo opuesto a lo anterior. Para hacer un solo cambio, tienes que modificar "a tiros" muchas clases diferentes.

*

****Feature Envy (Envidia de funcionalidad):**** Un método usa más datos o métodos de **otra** clase que de la suya propia.

*

****Refused bequest (Legado rechazado):**** Una clase hereda de otra pero apenas usa sus métodos/atributos (error de jerarquía).

3. Buenas Prácticas y Convenciones de Código

*** **Nomenclatura (Naming):****

*

***Clases / Interfaces:** ``UpperCamelCase`` (Primera letra mayúscula).

*

***Métodos / Variables:** ``lowerCamelCase`` (Primera letra minúscula).

*

***Constantes:** ``CONSTANT_CASE`` (Todo mayúsculas separadas por ``_``).

*

***Paquetes:** Todo en minúsculas.

*

****Strings y Objetos:**** Evitar instanciar con ``new`` si no es necesario. Usar tipos primitivos es mejor que clases Wrapper (envoltorio). ***Importante:** Nunca comparar objetos (como Strings o Wrappers) con ``==``, ya que esto compara direcciones de memoria, no el contenido.

*** **Magic Numbers:**** Son números o textos "mágicos" incrustados en el código sin explicación. Deben sustituirse por constantes descriptivas.

4. Analizadores de Código y TDD

*

****Análisis estático:**** Evalúa el código fuente ****sin llegar a ejecutarlo****.

*

****Herramientas clave:**** ****PMD**** (busca posibles bugs, código muerto, expresiones lógicas complejas) y ****CPD**** (forma parte de PMD y busca *código duplicado*).

*

****TDD (Test Driven Development / Desarrollo guiado por pruebas):**** Su característica principal (y típica pregunta de test) es que el programador ****implementa las pruebas ANTES de escribir el código**** de la unidad a probar.

5. Control de Versiones (CVS / SVN) y Configuración

*

****Repositorio:**** Es el componente principal; el lugar o directorio de almacenamiento central donde se guardan los datos, versiones y la evolución de los proyectos.

* ****Terminología típica de grafos de evolución:****

*

Tronco (Trunk): Variante principal.

*

Cabeza (Head): La última versión del tronco.

*

Ramas (Branches): Variantes secundarias (líneas de desarrollo paralelas).

*

Delta: El cambio concreto de una revisión respecto a la anterior.

* ****Comandos básicos:****

*

`checkout`: Obtener una copia local de trabajo desde el repositorio.

*

`update`: Actualizar tu copia local con los cambios del repositorio.

*

`commit`: Almacenar tu copia local modificada enviándola al repositorio.

*

****Gestión de la Configuración de Software (GCS) - Estándar IEEE 828:**** Sus 4 tareas básicas son: 1. Identificación, 2. Control de cambios, 3. Auditorías de configuraciones, 4. Generación de informes.

*

Nota de herramientas:** *Bugzilla**** sirve para seguimiento de bugs (bug-tracking) , mientras que CVS, Subversion (SVN), Git y Mercurial son de Control de Versiones.

6. Documentación (JavaDoc)

*

****Tipos de comentarios:**** Línea `//` , Bloque `/* */` , y ****JavaDoc `/** */`** (es obligatorio que empiece y termine exactamente con esos caracteres para que lo procese la herramienta).

*** **Etiquetas (Tags) de JavaDoc para métodos y clases:****

*

`@author`: Autor de la clase.

*

`@version`: Versión y fecha.

*

`@param`: Para indicar cada uno de los parámetros de un método/constructor.

*

`@return`: Lo que devuelve un método (si no es void).

*

`@throws` / `@exception`: Las excepciones que puede lanzar el método.

*

`@see`: Para crear una referencia/enlace a otra clase o método.

*

`@deprecated`: Indica que el método está obsoleto y no se recomienda su uso.